

Kernighan-Lin bipartition

Kernighan-Lin improves an existing bipartition by swapping pairs across the cut

The idea

- Score each unlocked pair by swap gain
- Here the winning prefix is one swap: A with D, gain nine
- Pass one then finds nothing and KL stops at a local optimum for swap moves

Pseudocode

- KL pseudocode builds a locked swap sequence each pass
- Pair gain uses D-values minus twice the edge between the pair
- Open this module's examples file and find the Pseudocode section
- That written sketch is what you implement on the implement track and what the browser

Algorithm sketch

- From the cutsize-twelve bad seed the winning prefix is one swap, A with D
- The sketch lands on ABC versus DE at cut three, then the next pass stops

Algorithm sketch — try these

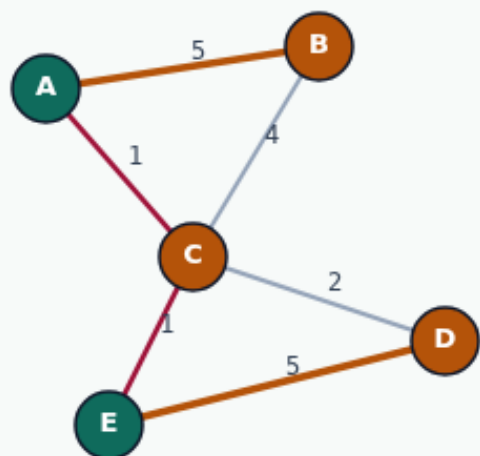
```
INPUT: bipartition side[], max_passes
OUTPUT: refined side[]
each pass: unlock all; compute D-values
repeat |V|/2: pick unlocked pair max swap gain
    lock pair; update working sides + D
keep prefix with best cumulative gain (>0)
apply prefix; stop if no improving pass
GOLDEN BAD_SEED cut 12 → ABC|DE cut 3
```

Bad seed: cutsize 12

KERNIGHAN-LIN BIPARTITION

Bad seed: cutsize 12

Step 1 / 5 · bad-seed · module02-01-kl-partition



Explanation

Start from a terrible bipartition $AE|BCD$. Both heavy edges $A-B$ and $D-E$ are cut, so cutsize is 12. KL will search improving swaps.

- Seed parts: A, E vs B, C, D
- Cut includes $A-B(5)$ and $D-E(5)$
- Goal: reduce cut without enumerating all partitions

```
seed:
{"A": "0", "E": "0", "B": "1", "C": "1", "D": "1"}
cutsize: 12
```

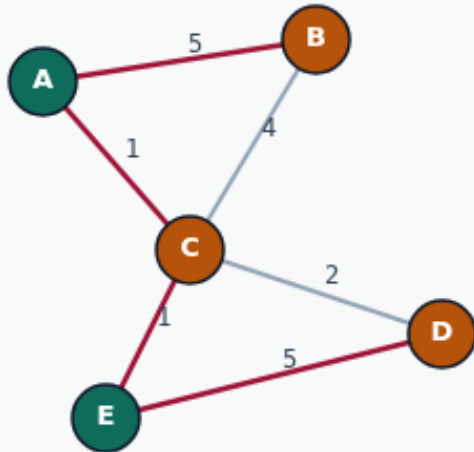
Bad seed: cutsize 12

Score pairwise swaps by gain

KERNIGHAN-LIN BIPARTITION

Score pairwise swaps by gain

Step 2 / 5 · gain-idea · module02-01-kl-partition



Explanation

KL considers swapping one vertex from each side. Gain estimates how much the cut shrinks. The best unlocked pair here is $A \leftrightarrow D$ with gain 9.

- $D(v) = \text{external} - \text{internal}$ for each vertex
- Swap gain uses D values and the edge between the pair
- Lock pairs after considering them in a pass

Best candidate swap: $A \leftrightarrow D$
gain $g = 9$

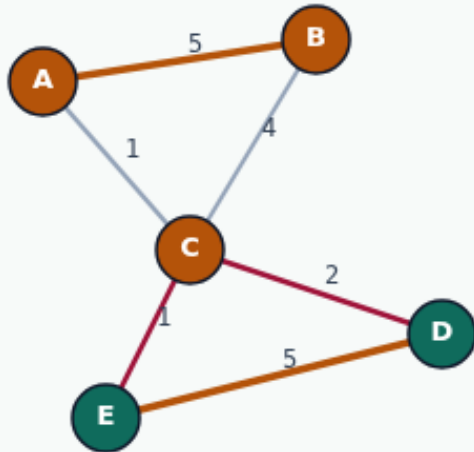
Score pairwise swaps by gain

Accept prefix: only $A \leftrightarrow D$

KERNIGHAN-LIN BIPARTITION

Accept prefix: only $A \leftrightarrow D$

Step 3 / 5 · accept-swap · module02-01-kl-partition



Explanation

Pass 0 builds a sequence of candidate swaps, then keeps the prefix with best cumulative gain. Here $\text{best_k}=1$: perform $A \leftrightarrow D$ once.

- $\text{best_k} = 1$
- $\text{bestCum} = 9$
- cut $12 \rightarrow 3$ in one swap

```
pass 0: A/D(9)
cutBefore=12 cutAfter=3
improved=true
```

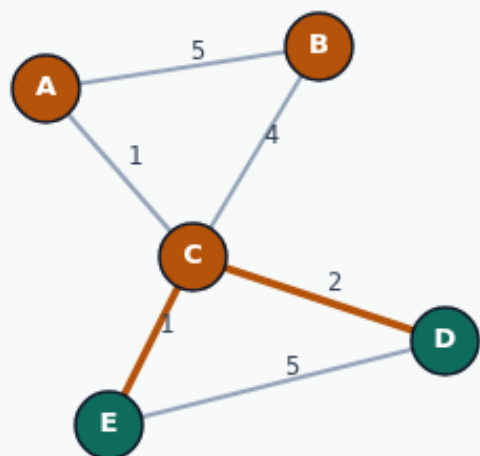
Accept prefix: only $A \leftrightarrow D$

Refined partition ABC|DE

KERNIGHAN-LIN BIPARTITION

Refined partition ABC|DE

Step 4 / 5 · final · module02-01-kl-partition



Explanation

After the swap, A joins B,C and D joins E. Heavy edges are internal; only the weak C-D/C-E bridge remains cut.

- Final parts: ABC|DE
- Matches the greedy/LP communities
- Cutsizes golden: 3

```
cutsizes: 3  
parts: ABC|DE
```

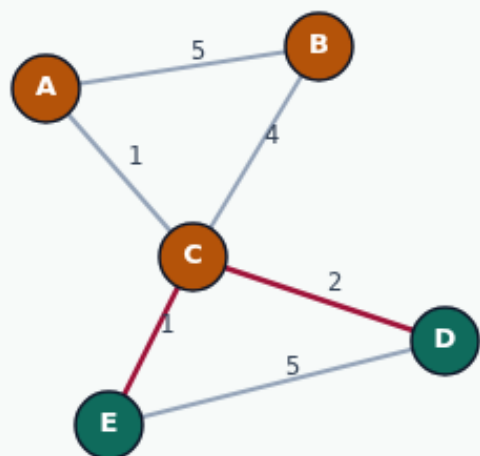
Refined partition ABC|DE

Next pass finds nothing

KERNIGHAN-LIN BIPARTITION

Next pass finds nothing

Step 5 / 5 · pass1-stop · module02-01-kl-partition



Explanation

Pass 1 reports improved=false. KL stops when a full pass cannot improve — local optimum for swap moves from this seed.

- Local, not global, optimum
- Quality depends on the seed
- Still the classic bipartition refiner

```
pass 1: best_k=0 improved=false  
stop
```

Next pass finds nothing

Browser lab track

- In the browser lab track, open the **kl-partition** lab from the tools shelf
- Load the starter graph, run the algorithm once
- Work the challenges that lock the goldens

Implement track

- In the implement track
- Parse the tiny graph, run the algorithm with a deterministic seed
- Match the browser goldens before you claim the checklist

Pitfalls

- Common traps
- For multilevel flows, verify coarsening before you blame the refiner

Your turn

- Complete the checklist for at least one track, preferably both
- Implement until your metrics match the starter goldens
- When you're ready, take the short quiz, then continue to the next module